DROPLET

B2B TRADE PORTAL

Trade ordering at your customer's contract price, on the web and the desktop.

Demo overview and how it works: the three apps, who uses what, a step-by-step demo script, the pricing, credit, sync and security model, and how a new business is set up and deployed.

Version 0.3 Web portal · Admin portal · Windows desktop app October 2026

What DROPLET is

DROPLET is a private ordering portal that a supplier (a wholesaler or distributor) runs for its approved trade customers. Customers see only their own negotiated prices, order by SKU in seconds, request quotes, reorder, and check their statement and credit. The supplier's staff approve customers, manage pricing and credit, and work every order and quote from an admin portal. A signed Windows desktop app gives trade counters an offline catalogue and instant SKU entry, protected by tamper detection.

Customers

Trade portal (web)

- Contract prices and quantity breaks
- Quick order by SKU, paste lists, reorder
- Quotes: request, then accept the offer
- Credit, aged balance and statement
- Get the signed desktop installer

Supplier staff

Admin portal

- Approve, reject, suspend applications
- Credit limits, terms, tiers, contract prices
- Approve, confirm, ship, invoice orders
- Price quotes; order for a customer
- Team, security, audit, live notifications

Trade counters

Desktop app (Windows)

- Same screens as the portal
- Local catalogue and prices: instant, offline
- Orders queue offline, send on sync
- Several trade accounts per computer
- App lock: password + authenticator

Who uses what

ROLE	WHERE	WHAT THEY CAN DO
Operator (you, DROPLET)	Server / command line	Deploy an install per business, generate its one-time setup link, publish desktop releases.
Administrator	Admin portal (web or desktop)	Everything: accounts, pricing, catalogue, orders, quotes, statements, team, security, settings.
Account manager	Admin portal (web or desktop)	Only the trade accounts assigned to them: their orders, quotes and ordering on their behalf.
Trade buyer	Trade portal and desktop app	Order, quote, reorder and view statements for their own account.

> Every staff and customer account that can install the desktop app or touch admin functions must turn on **two-step verification** (authenticator app). It is what allows recovery after a security lock and what protects the desktop app lock.

Demo script (about 20 minutes)

Run it on a fresh install. Have an authenticator app on your phone. Two browser windows help: one normal (admin), one private (customer).

1. **Create the business's first administrator.**
   Open the one-time setup link (printed in the server log on first start, or `npm run setup-link`). Enter the business name, your name, email and password. Sign in and scan the QR code: the admin portal opens.

2. **Build a small catalogue.**
   Admin → *Catalogue* → *Add product* (SKU, name, unit, list price, stock). Add a tier such as "Gold" and give one product quantity breaks for it.

3. **A customer applies.**
   In the private window open the portal → *Apply for an account*. Signing in now shows "Your account is under review". In the admin window the bell lights up with a pop-up: *New account application*.

4. **Approve with terms.**
   Admin → *Trade accounts* → the application → choose the Gold tier, a credit limit, payment terms and an account manager → *Approve account*. Optionally add a contract price for one SKU on *Contract prices*.

5. **The customer orders.**
   Customer signs in (the dashboard asks them to turn on two-step verification). *Quick order*: type a SKU, Enter, quantity, Enter. Prices shown are their own (contract → tier → list). Place an order larger than the limit to see it go for approval.

6. **Work the order.**
   Admin gets a *Order needs approval* notification → *Orders* → Approve → Confirm → Mark shipped → Invoice. Invoicing posts the amount to the customer's statement; their credit gauge and aged balance update.

7. **Quote round-trip.**
   Customer: *Request quote*. Admin: *Quotes* → price each line, set validity → *Send offer*. Customer: *Quotes* → *Accept* → it becomes an order at the offered prices.

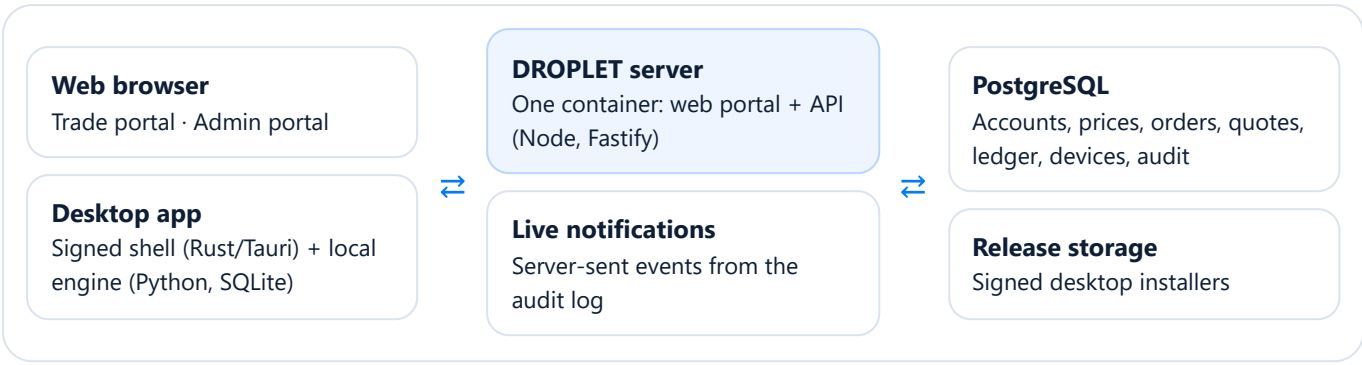
8. **Order for the customer.**
   Admin → the account → *New order*: the same quick-order grid with the customer's prices and credit check; the customer is emailed.

9. **Desktop app.**
   Customer → *Desktop app* → *Get signed installer* (single-use link + enrollment code). Install, paste the code, name the computer: catalogue and orders sync. Disconnect the network: ordering still works and queues. Use the padlock to lock; unlocking needs the password and then the authenticator code.

10. **Staff on desktop.**
    On the desktop app's title bar, the shield opens the admin portal: staff sign in with email, password and code. Notifications are checked every 15 seconds.

11. **Security story (optional).**
    Modify a file inside the installed app's `daemon` folder and relaunch: the app refuses to start, the account locks, the owner is emailed a recovery link. Recovery = link + password + code → clean installer → account unlocks after the clean install checks in.

How it works



Pricing

For every line the server resolves the best price for the quantity: the customer's **contract price**, else their **tier price**, else the **list price**. Prices shown in browsers or the desktop are only a preview; the server always recalculates on submit, so a client can never order below its contract price.

Credit

Balance = sum of the statement (invoices minus payments and credit notes). If balance + order total exceeds the credit limit, the order is held as *Awaiting approval* for staff. With no limit set, every order is held. Invoicing an order adds it to the statement with the customer's payment terms.

Desktop sync

Sync is explicit (at launch and on the sync button), never continuous. Each sync first proves the install is genuine, then pulls catalogue and price changes since the last sync, pushes queued orders (safe to retry, never duplicated) and refreshes the account summary for offline viewing.

Accounts on a computer

Several trade accounts can live on one computer, each with its own key and cache. At every launch the app checks each one with the server: accounts removed, rejected or revoked on the portal disappear from the desktop automatically.

Desktop integrity and tamper response

Every release is hashed file by file and the list is signed with the release key. At launch, and every 15 minutes, the shell re-checks every file, including files that should not be there. A mismatch stops the app before the engine runs, and the shell sends a signed report to the server.

Active → Locked (tamper) → Recovery email sent → Identity verified (link + password + code) → Clean build issued → Active again after clean install checks in

While locked, every session is signed out and web, desktop and sync access stop. Administrators can lift a lock that was a confirmed false alarm; the action is audited.

Sign-in, sessions and locks

- **Web:** password, then authenticator code; sessions are secure cookies (12 hours), stored only as hashes.
- **Desktop (buyers):** the computer is linked once with a code from the portal; it then authenticates with its own key, kept encrypted by Windows. The app lock needs the account's password and authenticator code, checked by the server, and survives restarts.
- **Desktop (staff):** live sign-in to the admin portal; the session lives only in the app's memory.
- **Audit:** sign-ins, approvals, price changes, orders, unlocks and every security event are recorded.

Setting up a business that buys DROPLET

Each business gets its **own install**: its own server, database, admin portal and customers. Nothing is shared between businesses.

1. Deploy
New install on Coolify (Docker). Starts with no administrator.

→

2. Setup link
Printed in the logs on first start, or `setup-link`. One use, 72 h.

→

3. First admin
Business opens the link, creates their administrator, turns on 2-step.

→

4. Onboard
Invite staff (Team). Approve or open customer accounts.

Deploying with GitHub, Coolify and Docker

1. Push the repository to GitHub.
2. In Coolify: *New resource → GitHub repository*, build pack **Docker Compose** using `docker-compose.yml` (portal + PostgreSQL), or **Dockerfile** with a Coolify-managed PostgreSQL.
3. Set the environment variables (generator command in `.env.production.example`): `PORTAL_ORIGIN`, `POSTGRES_PASSWORD` (or `DATABASE_URL`), `DROPLET_SIGNING_SECRET`, `DROPLET_MFA_ENC_KEY`, optionally `DROPLET_OPS_TOKEN`.
4. Give the `droplet` service your domain on port **8080**; Coolify issues the HTTPS certificate.
5. Deploy. Migrations run automatically. Open the logs for the one-time admin setup link.

Publishing the desktop app to a deployed install

```
npm --prefix apps/desktop run build -- -Api https://portal.example.com `
  -RegisterUrl https://portal.example.com -OpsToken <DROPLET_OPS_TOKEN>
```

The Windows build signs the release, points the app at your domain and uploads the installer to the server, where customers download it from their portal.

> **Before going live:** sign the installer with a code-signing certificate (otherwise Windows SmartScreen warns), keep the release signing key off the build machine (HSM/KMS), connect an email service (emails currently print to the server log), and back up `DROPLET_MFA_ENC_KEY` (losing it means everyone re-enrolls their authenticator).
